

Iteratieve statements

INFO

Lesinhoud

Controlestructuren: iteratieve statements.

Doelstellingen

De student(e) kan:

- de basisprincipes van JavaScript-programmatie (variabelen en constanten, controlestructuren, functies, arrays, objecten, DOM en DOM-manipulatie, events en event handling) toepassen binnen een project;

▼ Inhoud

- [Controlestructuren](#)
- [Iteratieve statements](#)
 - [Gesloten iteratieve statements](#)
 - [Open iteratieve statements](#)
- [Belangrijke punten](#)
- [Iteratie en objecten/arrays](#)
 - [Over de eigenschappen van een object itereren met for...in](#)
 - [Over een array itereren \(lopen\)](#)
- [Conclusie](#)

Controlestructuren

Eerder werden de `conditionele statements` behandeld, hier werden ook de `vergelijkingsoperators` en `logische operators` besproken. In dit onderdeel worden de `iteratieve statements` behandeld, waarbij opnieuw de kennis van de `vergelijkingsoperators` en `logische operators` toegepast wordt.

Ter herhaling, er zijn twee soorten controlestructuren:

- `conditionele statements`: een stuk code wordt uitgevoerd **als** een bepaalde voorwaarde `true` is.
- `iteratieve statements`: een stuk code wordt herhaaldelijk uitgevoerd **zolang** een bepaalde voorwaarde `true` is.

Iteratieve statements

Iteratieve statements worden gebruikt om een stuk code herhaaldelijk uit te voeren.

Er zijn twee soorten iteratieve statements:

- gesloten iteratieve statements: de code wordt een vast aantal keren uitgevoerd (je weet op voorhand hoeveel keer de code uitgevoerd zal worden)
- open iteratieve statements: de code wordt uitgevoerd zolang aan een bepaalde voorwaarde wordt voldaan (je weet op voorhand niet hoeveel keer de code uitgevoerd zal worden)

Gesloten iteratieve statements

`for` -statement

Een `for`-statement of `for`-loop is een gesloten iteratieve statement. Dit betekent dat de code een vast aantal keren wordt uitgevoerd.

METAFOOR

De docent heeft een lijst met namen van studenten en wilt de aanwezigheid van elke student controleren. De docent gaat de lijst één voor één af en controleert of de student aanwezig is. Dit is een gesloten iteratieve statement, omdat de docent weet hoeveel studenten er zijn en dus weet hoe vaak hij de actie moet uitvoeren.

Stel je voor dat er tien studenten zijn. De docent zal tien keer de actie uitvoeren om de aanwezigheid van elke student te controleren.

De syntax is als volgt:

```
for (initialisatie; voorwaarde; iteratie) {  
  // code die wordt uitgevoerd  
}
```

js

Concreet voorbeeld:

```
for (let teller = 0; teller < 5; teller++) {  
  console.log("Dit is iteratie nummer " + teller);  
}
```



```
/**  
 * Uitleg:  
 * for-statement waarbij er een variabele genaamd teller  
wordt geïnitieerd met de waarde 0.  
 * De voorwaarde om het codeblok uit te voeren is dat de  
waarde van teller kleiner is dan 5.  
 * Na elke iteratie wordt de waarde van teller met 1  
verhoogd.  
 * Het codeblok wordt dus 5 keer uitgevoerd.  
 * Output:  
 * Dit is iteratie nummer 0
```

js

```
* Dit is iteratie nummer 1
* Dit is iteratie nummer 2
* Dit is iteratie nummer 3
* Dit is iteratie nummer 4
*/
```

Open iteratieve statements

while -statement

Een **while** -statement of **while** -loop is een open iteratieve statement. Dit betekent dat de code wordt uitgevoerd zolang aan een bepaalde voorwaarde wordt voldaan.

METAFOOR

Bij het spel **een, twee, drie piano (Vlaanderen)**, **Annemarie Koekoek (Nederland)** of **red light, green light** is er een persoon die het spel leidt die zich ofwel naar de spelers richt - ofwel van de spelers weggijkt. De andere personen moeten proberen zo snel mogelijk naar de leider toe te rennen, maar mogen niet bewegen als de leider zich naar hen richt. Dit is een open iteratieve statement, een speler blijft rennen zolang de leider weggijkt, maar stopt als de leider zich naar hen richt. De speler weet op voorhand niet hoe lang er gerend moet worden.

De syntax is als volgt:

```
while (voorwaarde) {
  // code die wordt uitgevoerd
}
```

js

Concreet voorbeeld:

Om het voorbeeld uit te leggen gaan we gebruik maken van een 'Math.random()'-functie die een willekeurig getal tussen 0 en 1 genereert (bijvoorbeeld `0.14143224237383178` of `0.7155762022634901`). Wanneer dit willekeurig getal tussen de range 0.45 en 0.55 ligt, dan stoppen we met het uitvoeren van de code.

```
js
let getalInRange = false;

while (getalInRange !== true) {
  const willekeurigGetal = Math.random(); // Genereert
  een willekeurig getal tussen 0 en 1, bijvoorbeeld 0.33,
  0.67, 0.5, etc.
  console.log("Het gegenereerde getal is: " +
  willekeurigGetal);

  if (willekeurigGetal >= 0.45 && willekeurigGetal <=
  0.55) {
    getalInRange = true;
  }
}

/**
 * Uitleg:
 * We initialiseren een boolean variabele genaamd
  getalInRange met de waarde false.
 * We gebruiken een while-loop die blijft uitgevoerd
  worden zolang getalInRange false is.
 * In elke iteratie van de loop genereren we een
  willekeurig getal tussen 0 en 1.
 * We controleren of het willekeurig getal tussen 0.45 en
  0.55 (inclusief 0.45 en 0.55) ligt.
 * Als dat het geval is, zetten we getalInRange op true,
  waardoor de loop stopt.
 * Output: De willekeurige getallen worden afgedrukt
  totdat er een getal in de range ligt.
 */
```

Dit zal voor iedereen andere resultaten opleveren, omdat de `Math.random()` -functie een willekeurig getal genereert. Het kan zijn dat het eerste getal al in de range ligt, maar het kan ook zijn dat er meerdere iteraties nodig zijn om een getal in de range te genereren.

Hieronder een drie voorbeelden van mogelijke uitvoer:

```
Het gegenereerde getal is: 0.06446777692065775
Het gegenereerde getal is: 0.34436778222272224
Het gegenereerde getal is: 0.0628674443300492
Het gegenereerde getal is: 0.3831347423353173
Het gegenereerde getal is: 0.8824160787830156
Het gegenereerde getal is: 0.5249911572728416
```

```
Het gegenereerde getal is: 0.5066959737836711
```

```
Het gegenereerde getal is: 0.21079089592494715
Het gegenereerde getal is: 0.12846476800766504
Het gegenereerde getal is: 0.4462960247565988
Het gegenereerde getal is: 0.7772044634140478
Het gegenereerde getal is: 0.3495799218233083
Het gegenereerde getal is: 0.8615680053030249
Het gegenereerde getal is: 0.4598447011258773
```

`do while` -statement

Er is ook een `do while` -statement of `do while` -loop, dit is een open iteratieve statement die altijd minstens één keer wordt uitgevoerd, ongeacht of de voorwaarde `true` of `false` is. De syntax is als volgt:

```
do {
  // code die wordt uitgevoerd
```

js

```
} while (voorwaarde);
```

Concreet voorbeeld:

```
let invoer;
const geheim = "hunter2";

do {
  invoer = prompt("Geef het geheim in:");
} while (invoer !== geheim);

console.log("Geheim gevonden!");

/**
 * Uitleg:
 * We declareren een variabele genaamd invoer.
 * We initialiseren een constante genaamd geheim met de
waarde "hunter2".
 * We gebruiken een do while-loop, het codeblok wordt
minstens één keer uitgevoerd.
 * In het codeblok vragen we de gebruiker om het geheim
in te voeren via een prompt.
 * Het codeblok blijft uitgevoerd worden de invoer niet
gelijk is aan het geheim.
 * Zodra de gebruiker het juiste geheim invoert, wordt
het uitvoeren gestopt en wordt "Geheim gevonden!"
afgedrukt.
 */
```

Dit is een open iteratieve statement omdat we niet op voorhand weten hoe vaak de gebruiker een foutieve invoer zal geven. De gebruiker moet het geheim invoeren, en zolang dit niet correct is, blijft er opnieuw gevraagd worden om het geheim in te voeren.

Belangrijke punten

Een `while`-loop kan ook gebruikt worden voor gesloten iteratieve statements, een `for`-loop kan niet gebruikt worden als open iteratieve statement. In dat geval moeten we een teller bijhouden.

```
let teller = 0;
while (teller < 5) {
  console.log("Dit is iteratie nummer " + teller);
  teller++;
}
```

js

Dit doet hetzelfde als:

```
for (let teller = 0; teller < 5; teller++) {
  console.log("Dit is iteratie nummer " + teller);
}
```

js

Een open iteratieve statement kan leiden tot een oneindige loop (infinite loop) als de voorwaarde nooit `false` wordt. Dit zal leiden tot een programma dat blijft draaien zonder te stoppen / de browser zal crashen. Het is belangrijk om ervoor te zorgen dat de voorwaarde ooit `false` wordt.

Voorbeeld van een oneindige loop:

```
let teller = 0;
while (teller < 5) {
  console.log("Dit is iteratie nummer " + teller);
  // teller wordt niet verhoogd, dus de voorwaarde blijft
  // altijd true, de loop blijft oneindig doorgaan
}
```

js

js

```
let getal = 0;
while (getal === 10) {
  console.log("Het getal is: " + getal);
  getal = getal + 3;
}

/**
 * Eerste iteratie: Het getal is 0
 * Tweede iteratie: Het getal is 3
 * Derde iteratie: Het getal is 6
 * Vierde iteratie: Het getal is 9
 * Vijfde iteratie: Het getal is 12
 * Zesde iteratie: Het getal is 15
 * ...
 * De loop blijft oneindig doorgaan, omdat we nooit op
 * getal === 10 uitkomen.
 */
```

Iteratie en objecten/arrays

Een `for`-loop en een `while`-loop kunnen gebruikt worden voor alle iteraties. Er zijn ook specifieke manieren om over objecten en arrays te itereren. Voor objecten gebruiken we de `for...in`-loop, voor arrays de `for...of`-loop.

Over de eigenschappen van een object itereren met `for...in`

Soms wil je alle eigenschappen van een object doorlopen, bijvoorbeeld om elke eigenschap en zijn waarde op de webpagina te tonen. Hiervoor kunnen we de `for...in` loop gebruiken. Deze structuur is specifiek ontworpen om door de eigenschappen van een object te gaan.

html

```
<ul id="student-info"></ul>
```

js

```
const student = {
  voornaam: "Jan",
  achternaam: "Janssens",
  opleiding: "Graduaat Programmeren",
};

// We selecteren het <ul>-element uit de HTML.
const studentInfoLijst =
document.querySelector("#student-info");

for (const key in student) {
  /*
   * 'key' is een variabele die bij elke iteratie
   * de naam van een andere eigenschap bevat.
   * Dus eerst "voornaam", dan "achternaam", etc.
   *
   * Merk op dat de []-notatie gebruikt wordt:
   * student['voornaam']
   * student['achternaam']
   */
  const value = student[key];

  /**
   * Maak een nieuw <li>-element aan.
   */
  const listItem = document.createElement("li");
  /**
   * Wijzig de tekst van het <li>-element.
   * Er wordt een `template literal` gebruikt (backticks
   * ``).
   */
  listItem.innerText = `${key}: ${value}`;
  /**
   * Onderstaande regel zou hetzelfde resultaat geven via
   * string-concatenatie.
```

```
*/  
//listItem.innerText = key + ": " + value;  
  
/**  
 * Voeg het <li>-element toe aan het <ul>-element.  
 */  
studentInfoLijst.appendChild(listItem);  
}
```

- voornaam: Jan
- achternaam: Janssens
- opleiding: Graduaat Programmeren

▼ Oefening

Voeg in bovenstaande code een extra eigenschap `leeftijd` toe aan het `student`-object.

Over een array itereren (lopen)

Vaak willen we alle elementen van een array doorlopen om er iets mee te doen. Om bijvoorbeeld iets te berekenen of om de gegevens op de webpagina te tonen. Dit kan op verschillende manieren.

De traditionele for-loop

De klassieke for-loop, die je kent van andere controlestructuren, is ook perfect bruikbaar voor arrays. Hierbij gebruik je een teller (vaak `i` genoemd) als index om elk element aan te spreken.

html

```
<ul id="dierenlijst"></ul>
```

js

```
const dieren = ["hond", "kat", "vogel"];
/*
 * Zoek het eerste element met id "dierenlijst".
 * Dit is een <ul> in dit voorbeeld.
 */
const lijst = document.querySelector("#dierenlijst");

/*
 * i start bij 0 (let i = 0;)
 *
 * het itereren (de loop) gaat door
 * zolang i kleiner is dan de lengte van de array
 * (i < dieren.length;)
 *
 * i wordt na elke stap met 1 verhoogd.
 * (i++)
 */
for (let i = 0; i < dieren.length; i++) {
  const dier = dieren[i]; // Haal het element op index i
  uit de array
  const li = document.createElement("li");
  li.innerText = `Het dier op index ${i} is een
  ${dier}.`;
  lijst.appendChild(li); // voeg de <li> als child
  element toe aan de <ul>
}
```

- Het dier op index 0 is een hond.
- Het dier op index 1 is een kat.
- Het dier op index 2 is een vogel.

▼ Oefening

Voeg een extra dier toe aan de `dieren`-array in bovenstaande code. Bekijk het resultaat.

Over elementen van een array itereren met `for...of`

Een meer moderne en leesbare manier om over de elementen van een array te itereren is met de `for...of`-loop.

```
<ul id="neefjes"></ul>
```

html

```
const neefjes = ["Kwik", "Kwek", "Kwak"];

// We selecteren het <ul>-element uit de HTML.
const studentInfoLijst =
document.querySelector("#neefjes");

for (const neef of neefjes) {
  /*
   * 'neef' is een variabele die bij elke iteratie
   * een element uit de array bevat.
   * Dus eerst "Kwik", dan "Kwek", etc.
   */

  /**
   * Maak een nieuw <li>-element aan.
   */
  const listItem = document.createElement("li");
  /**
   * Wijzig de tekst van het <li>-element.
   */
  listItem.innerText = neef;

  /**
```

js

```
* Voeg het <li>-element toe aan het <ul>-element.  
*/  
studentInfoLijst.appendChild(listItem);  
}
```

- Kwik
- Kwek
- Kwak

▼ Oefening

Wijzig de code zodat onderstaande op de webpagina verschijnt:

- Kwik is een neefje van Donald Duck
- Kwek is een neefje van Donald Duck
- Kwak is een neefje van Donald Duck

Conclusie

De iteratieve statements worden gebruikt om een stuk code herhaaldelijk uit te voeren. Er zijn twee soorten iteratieve statements: gesloten en open iteratieve statements. Gesloten iteratieve statements worden gebruikt wanneer we weten hoeveel keer de code moet worden uitgevoerd, terwijl open iteratieve statements worden gebruikt wanneer we niet weten hoeveel keer de code moet worden uitgevoerd.

Het is mogelijk om open iteratieve statements zo op te bouwen dat ze ook gesloten iteratieve statements worden. Het is belangrijk om ervoor te zorgen dat de voorwaarde ooit `false` wordt, anders kan dit leiden tot een oneindige loop.

Previous page
Oefeningen

Next page
Oefeningen